

Parser Combinators

Exam syntax cheat sheet · Functional Programming (F#)

1. Grammar notation (BNF)

A context-free grammar rewrites non-terminals into strings of terminals/non-terminals. Shorthand BNF form groups alternatives with |:

```
A ::= N | V | A + A | A * A | ( A ) | - A
```

Left-recursive (BAD - infinite loop risk, needs lookahead):

```
T ::= T + T | P
```

Right-recursive rewrite (fixes left recursion):

```
T ::= P + T | P
```

Precedence, low -> high: T (terms, +) -> P (products, *) -> A (atoms)

Rule of thumb: rewrite left recursion to right recursion; encode precedence by layering non-terminals (lowest precedence outermost).

2. The Parser type

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
(* refined version used later in the course, with position tracking: *)
type Parser<'a> = TextInputState -> Result<'a * TextInputState, ParseError>
type ParseError = { label: string; input: TextInputState; error: string }
type Result<'a, 'b> = | Ok of 'a | Error of 'b
```

A parser is a **function**: input → (parsed value, remaining input) or an error. Every combinator below is just a function that builds a new such function out of existing ones.

3. Running a parser

```
run : Parser<'a> -> string -> Result<'a, string>
```

```
let run p str =
  match p (inputStr str) with
  | Ok (value, _) -> Ok value
  | Error err      -> Error (errorStr err)
```

4. Primitive parsers

Syntax	Meaning
pchar c	matches exactly the character c
satisfy pred	matches next char if pred holds; base for pchar etc.
anyChar	= satisfy (fun _ -> true) - matches any single char
pdigit	= satisfy Char.IsDigit
pletter	= satisfy Char.IsLetter
pint	= many1 pdigit >> (implode >> int) - parses an integer
spaces	= many (satisfy Char.IsWhiteSpace) - zero or more whitespace

```
let pchar (ch: char) : Parser<char> = fun str ->
  if str.Length = 0 then Error "No more input"
  else
    let first = str.[0]
```

```

    if first = ch then Ok (first, str.[1..])
    else Error (sprintf "Expecting '%c'. Got '%c'" ch first)

let satisfy (pred: char -> bool) : Parser<char> = fun str ->
  if str.Length = 0 then Error "No more input"
  else
    let first = str.[0]
    if pred first then Ok (first, str.[1..])
    else Error (sprintf "Unexpected '%c'" first)

let pchar c = satisfy (fun x -> x = c)

```

5. Core combinators

Syntax	Meaning
>>	Parser<'a> -> ('a -> 'b) -> Parser<'b> — map the result
.>>.	Parser<'a> -> Parser<'b> -> Parser<'a*b> — sequence, keep both
.>>	Parser<'a> -> Parser<'b> -> Parser<'a> — sequence, keep left
>>.	Parser<'a> -> Parser<'b> -> Parser<'b> — sequence, keep right
< >	Parser<'a> -> Parser<'a> -> Parser<'a> — try p1, else p2
many	Parser<'a> -> Parser<'a list> — zero or more (never fails)
many1	Parser<'a> -> Parser<'a list> — one or more (fails on zero)
<?>	Parser<'a> -> string -> Parser<'a> — attach error label
between l p r	= l >>. p .>> r — parse p surrounded by l and r, keep p
createParserForwardedToRef	unit -> Parser<'a> * Parser<'a> ref — for recursive parsers

```

let (.>>.) p1 p2 = fun inp ->
  match p1 inp with
  | Error err -> Error err
  | Ok (val1, rem1) ->
    match p2 rem1 with
    | Error err -> Error err
    | Ok (val2, rem2) -> Ok ((val1, val2), rem2)

let (|>>) p f = fun inp ->
  match p inp with
  | Ok (value, rem) -> Ok (f value, rem)
  | Error err -> Error err

let (.>>) p1 p2 = (p1 .>> . p2) |>> fst
let (>>.) p1 p2 = (p1 .>> . p2) |>> snd

let (<|>) p1 p2 = fun inp ->
  match p1 inp with
  | Ok result -> Ok result
  | Error _ -> p2 inp

let rec repeat p inp =
  match p inp with
  | Error _ -> ([], inp)
  | Ok (value, rem1) ->
    let (values, rem2) = repeat p rem1
    (value :: values, rem2)
let many p = fun inp -> Ok (repeat p inp)
let many1 p = (p .>> . many p) |>> (fun (v, vs) -> v :: vs)

let between l p r = l >>. p .>> r

let (<?>) p l = fun input ->
  match p input with

```

```
| Ok s      -> Ok s
| Error pe -> Error { pe with label = l }
```

6. Recursive parsers (forward reference trick)

F# only allows **functions** to be defined with **let rec ... and** Parser *values* that refer to each other need a forward reference instead:

```
let pT, tref = createParserForwardedToRef<aexp>()
let pP, pref = createParserForwardedToRef<aexp>()
let pA, aref = createParserForwardedToRef<aexp>()

let pN  = pint |>> N
let pV  = pid  |>> V
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces) .>>. pT) |>> Add
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces) .>>. pP) |>> Mul

do tref := pAdd <|> pP
do pref := pMul <|> pA
do aref := pNeg <|> pPar <|> pN <|> pV
```

pT/pP/pA can be referenced immediately when defined; the ref just needs to be filled in with 'do ... := ...' before run is called.

7. Building your own parser - general recipe

1. Identify the smallest pieces to recognise (chars, digits, keywords...).
2. Write/reuse primitive parsers for them (pchar, satisfy, pdigit...).
3. Combine in the shape of the input:
 - "X then Y" -> .>>. / .>> / >>.
 - "zero/one or more"-> many / many1
 - "X or Y" -> <|>
 - skip whitespace -> wrap with spaces via .>> / >>.
4. Map the raw parsed shape into the value you actually want: |>> f
5. If recursive, use createParserForwardedToRef + do ref := ...

Worked example: identifier

```
let pidentChar = satisfy (fun c -> System.Char.IsLetterOrDigit c || c = '_')
let pident : Parser<string> =
    (pletter .>>. many pidentChar)
    |>> (fun (first, rest) -> implode (first :: rest))
```

Worked example: key: value pair

```
let pKeyValue : Parser<string * string> =
    (pident .>> spaces .>> pchar ':' .>> spaces) .>>. pident
```

8. Common F# helper functions

Syntax	Meaning
explode str	string -> char list — [for c in str -> c]
implode cs	char list -> string — cs > Array.ofList > System.String
String.concat sep list	joins a string list with separator sep
str.Split(' ')	string -> string[] — splits on the given char
List.chunkBySize n list	splits a list into groups of (at most) size n, in order
Array.toList arr	converts array to list (read array without mutating it)

```
let explode (str: string) = [for c in str -> c]
let implode (cs: char list) = cs |> Array.ofList |> System.String
```

```
(* chunkBySize needs a SIZE, not a target count - convert with ceiling division: *)
let chunkSize = (n + num - 1) / num          // n = total items, num = wanted chunks
let chunks = List.chunkBySize chunkSize words
```

9. Parallelism syntax (Tasks)

```
System.Threading.Tasks.Task.Run(fun () -> ...) // starts work immediately, returns a Task<'a>
task.Result                                     // blocks until done, retrieves the result

let tasks =
  chunks
  |> List.map (fun chunk ->
    System.Threading.Tasks.Task.Run(fun () ->
      chunk |> List.map encode |> String.concat " "))

let results = tasks |> List.map (fun t -> t.Result) // collect, preserving order
```

Each task must work on an INDEPENDENT piece of data (e.g. one word does not depend on another) for parallelising to be safe and to give the same result as the sequential version.

10. Quick reference: combinator cheat table

Syntax	Meaning
<code>pN = pint >> N</code>	wrap a primitive result into your AST type
<code>pchar '-' >>. spaces >>. pA</code>	consume '-' and spaces, keep only pA's result
<code>between (pchar '(') pT (pchar ')')</code>	parse pT surrounded by parens, discard parens
<code>(pP .>>. pT) >> Add</code>	sequence two parsers, then combine into an AST node
<code>pNeg < > pPar < > pN < > pV</code>	try each alternative left-to-right
<code>many1 pletter >> implode</code>	one-or-more letters, joined into a string
<code>p "integer"</code>	give this parser a readable name for error messages